

UserRepository and declares the additional function *findOne()*.

```
public interface
userRepositoryDecorator extends
userRepository {
    public Optional<String>
findOne(Long phone);
}
```

A class like *FluentUserRepository* can be an implementation of the *UserRepositoryDecorator*.

```
public class FluentUserRepository
implements userRepositoryDecorator {
    private userRepository target;

    public FluentUserRepository
(userRepository target) throws
Exception {
        this.target = target;
    }
    ...
}
```

The consumer of the decorator now refers to the interface, instead of the concrete class directly.

```
userRepositoryDecorator fluent = new
FluentUserRepository(repo);
fluent.findOne(9731423166L).ifPresent(name -> System.out.
println("Found " + name));
```

Interface segregation

Let us try to understand where we use the decorator pattern.

One of the dilemmas for a designer is whether to offer all functionalities in one interface or split them into individual interfaces. The principle of *Single Responsibility* recommends not to mix multiple features into a single interface. Usually, the core interface is defined only with cohesive mandatory use cases, whereas the optional use cases are handled in individual proxies and decorators. Proxies are useful when the additional use cases can be plugged-in just by adding pre-processing and post-processing. However, sometimes, the additional functionality may require additional interfaces altogether for the consumers to invoke. Such interfaces and their implementations are called decorators.

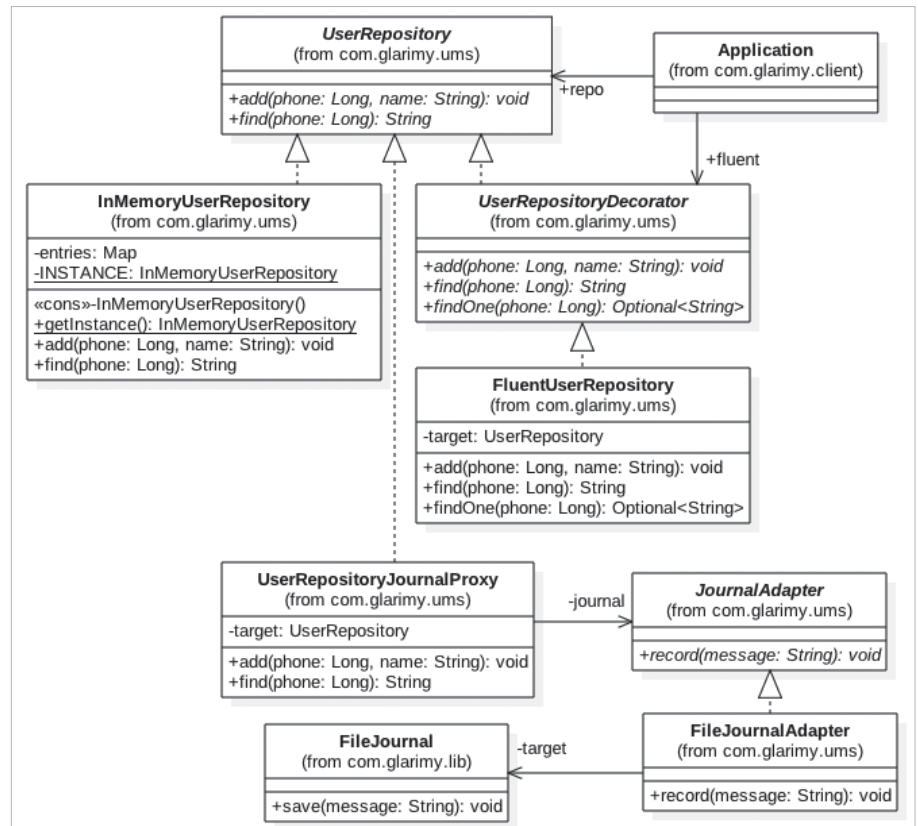


Figure 5: Modular UMS design

Finally, our refactored design for UMS is more modular with proxies, decorators, and adapters. This is depicted in Figure 5.

The best practices learnt in this part can be enumerated like this:

1. Design an adapter between your code and third party modules. It protects your code from changes that happen in the third party modules.
2. Design proxies for different pluggable features.
3. Design decorators for extending the functionality of an object.
4. Design the factory to load the adapters, proxies, etc.

Almost all modern systems are modular. Other patterns like 'composite', 'facade', etc, are also useful in building modular systems.

In the next part, the focus will be on establishing communication among modules. **END** 🐼

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.